Chapter 1: Introduction

Algorithms for Optimization

2nd Edition (2026)

Kochenderfer & Wheeler

MIT Press

2026

Chapter Overview

Outline

A Brief History of Optimization I

Optimization is one of the oldest intellectual pursuits in mathematics. Long before computers existed, thinkers were asking: "What is the *best* arrangement, shape, or strategy?" Understanding this history helps us see why the mathematical tools we use today look the way they do, and why they are so effective across so many different fields.

Ancient Roots (300–250 BC)

The Greeks posed geometric extremum problems — questions such as "which shape encloses the largest area for a fixed perimeter?" **Euclid** (∼300 BC) established rigorous geometric reasoning, and showed that among all rectangles with a given perimeter, the square has the largest area. **Archimedes** (∼250 BC) studied the problem of maximising the area of a polygon inscribed in a circle. As the number of polygon sides grows without bound, the polygon area approaches the circle area — a conceptual forerunner of the *calculus of variations*, which optimises over *shapes* rather than just numbers. Much later, **Fibonacci** (1170–1250) popularised algebra and the idea that mathematics could be applied to solve practical problems of commerce and engineering, planting seeds that would eventually grow into modern optimization theory.

A Brief History of Optimization II

The Calculus Era: 17th and 18th Centuries

The invention of calculus made it possible to study *rates of change* precisely. This turns out to be exactly what is needed to find maxima and minima — because at a maximum or minimum, the rate of change of the function must be zero.

Fermat (1643) observed that the derivative of a smooth function must equal zero at an interior maximum or minimum. This single insight — "set the derivative to zero and solve" — remains the cornerstone of optimization theory to this day. Every gradient-based algorithm in this book is, at its heart, searching for a point where the derivative vanishes.

Newton and Leibniz (1660s–1680s) independently developed differential calculus, providing the machinery to compute derivatives systematically and to identify *stationary points* of arbitrary functions. Newton also gave us an iterative root-finding procedure (Newton's method) that, when applied to the derivative, directly finds stationary points; this is the basis of Newton's optimization method in Chapter 6.

Euler and Lagrange (1740–1770s) extended these ideas to *functions of functions*, founding the *calculus of variations*. Lagrange (pronounced "la-GRON-zh") also introduced his celebrated *multiplier method* for handling equality constraints. In Lagrange's method, a new variable λ (lambda) is introduced for each constraint, and the constrained problem is converted into an unconstrained one

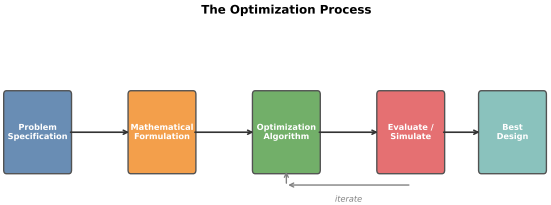
Outline

The Optimization Process I

Optimization is not just a mathematical operation — it is a *process* that begins with a real-world problem and ends with an implementable design. Understanding this process prevents the common mistake of jumping straight to an algorithm without first carefully formulating the problem.

The Five-Step Engineering Cycle

1. **Specify the design problem.** What physical or business system are you trying to improve? What can you actually change (the *design variables*)? What outcome do you care about (the *objective*)? Being precise here prevents wasted effort later.
2. **Choose a mathematical formulation.** Translate the problem into a precise mathematical form: define the objective function $f(\mathbf{x})$ (f of x), the design vector $\mathbf{x}$ (bold x), and any constraints.
3. **Select an optimization algorithm.** Different algorithms suit different problem



Representative Design Problems

- **Aerospace**: choose airfoil chord (front-to-back length), thickness, and sweep angle to minimise aerodynamic drag subject to minimum-lift and structural-integrity constraints.

The Optimization Process II

How to Choose an Algorithm

Before selecting an algorithm, you must characterise your problem along several dimensions. There is no universally best choice — the right algorithm depends on problem structure. Here are the key questions to ask.

Dimensionality (n, the number of design variables): low-dimensional problems ($n < 10$) allow exhaustive or derivative-free search, while high-dimensional problems ($n > 1000$) typically require gradient-based methods because they scale much better with n.

Gradient availability: some problems supply exact gradients analytically or via automatic differentiation (Chapters 2–3); others only allow function evaluations, requiring derivative-free methods (Chapter 4 onward).

Constraint structure: unconstrained problems are simpler to handle; bound constraints (Chapter 4), equality constraints, and general inequality constraints (Chapter 10) each require specialised treatment.

Evaluation budget: if each evaluation of $f(\mathbf{x})$ requires an hour-long Computational Fluid Dynamics (CFD) simulation, you can afford far fewer iterations than if $f(\mathbf{x})$ evaluates in microseconds.

Noise and stochasticity: if $f(\mathbf{x})$ is corrupted by random noise (e.g., measured experimentally), stochastic optimizers (Chapters 8–9) are more appropriate than deterministic gradient descent.

Outline

Mathematical Formulation of an Optimization Problem I

Before an algorithm can be applied, the real-world problem must be translated into a precise mathematical statement. This section introduces the standard form used throughout the book. Every optimization problem in this text can be written in this form, so it is worth spending time to understand it thoroughly.

Standard Form of an Optimization Problem

An optimization problem is written as:

$$\min_{\mathbf{x}\in\mathcal{X}} \; f(\mathbf{x}) \quad \text{subject to} \quad g_i(\mathbf{x}) \leq 0, \; i = 1, \ldots, m, \quad h_j(\mathbf{x}) = 0, \; j = 1, \ldots, p$$

Mathematical Formulation of an Optimization Problem II

Notation: Every Symbol Explained in Plain Language

- $\mathbf{x} = (x_1, x_2, \ldots, x_n)^\top \in \mathbb{R}^n$ is the **design vector** (also called the *decision variable* or *optimisation variable*). It is a column vector containing n real numbers that the algorithm is free to choose. The superscript $\top$ (top) means "transpose" — turning a row of numbers into a column. Each component x_i (x sub i) is one design parameter: for example, in wing design, x_1 might be the chord length, x_2 the thickness-to-chord ratio, and so on. The notation $\mathbb{R}^n$ (blackboard R to the power n) denotes the set of all n-tuples of real numbers.
- $f : \mathbb{R}^n \to \mathbb{R}$ is the **objective function**. This notation means f takes an n-dimensional input vector $\mathbf{x}$ and returns a single real number, measuring how "good" that design is. We write *min* to indicate we want the smallest possible value; if you want to maximise, simply minimise $-f$.
- $g_i(\mathbf{x}) \leq 0$ for $i = 1, \ldots, m$ are the **inequality constraints**. There are m (m) of them. The subscript i indexes each constraint. Writing $g_i \leq 0$ is a universal convention: any constraint can be rewritten in this form. For example, "thrust $\geq$ 500 N" becomes $-\text{thrust}(\mathbf{x}) + 500 \leq 0$.
- $h_j(\mathbf{x}) = 0$ for $j = 1, \ldots, p$ are the **equality constraints**. There are p (p) of them. The subscript j indexes each one. For example, "total portfolio weight equals 1" becomes $\sum_{i=1}^{n} x_i - 1 = 0$, where $\sum$ (Sigma, meaning "sum") adds up all n weights.
- $\mathcal{X}$ (calligraphic X, read "script X") is the **feasible set**: the collection of all vectors $\mathbf{x}$ that

Feasible Set — Worked Numerical Example I

To make the abstract standard form concrete, let us work through a small two-dimensional example completely by hand. This illustrates how constraints shape the set of allowable solutions and why the constrained optimum may differ from the unconstrained one.

Feasible Set — Worked Numerical Example II

Example: A 2-D Constrained Problem

We wish to minimise the squared distance from the point $(2, 1)$:

$$\min_{x_1,\, x_2} \; f(x_1, x_2) = (x_1 - 2)^2 + (x_2 - 1)^2$$

subject to three constraints:

$$g_1 : \; x_1 + x_2 \leq 2, \qquad g_2 : \; x_1 \geq 0, \qquad g_3 : \; x_2 \geq 0.$$

Here x_1 (x sub 1) and x_2 (x sub 2) are the two decision variables. The objective f measures the squared Euclidean distance between the current point (x_1, x_2) and the fixed target point $(2, 1)$. The constraints g_2 and g_3 restrict us to the first quadrant (both coordinates non-negative), while g_1 cuts off the upper-right corner.

Feasible Set — Worked Numerical Example III

Step 1: Find the Unconstrained Minimum

Without any constraints, we minimise f by finding where its gradient is zero. Since $f = (x_1 - 2)^2 + (x_2 - 1)^2$ is a sum of squares, it achieves its minimum value of 0 exactly at the target point $(x_1, x_2) = (2, 1)$.

Check feasibility: does $(2, 1)$ satisfy the constraint g_1?

$$x_1 + x_2 = 2 + 1 = 3 \not\leq 2. \quad \text{Infeasible!}$$

The unconstrained optimum violates the first constraint, so we cannot simply ignore the constraints here. We must look for the best feasible point.

Feasible Set — Worked Numerical Example IV

Step 2: Examine the Active Boundary

Because the unconstrained minimum is outside $\mathcal{X}$, the constrained minimum must lie *on the boundary* of the feasible set. The boundary closest to $(2, 1)$ is the line $x_1 + x_2 = 2$ (where g_1 is active, meaning satisfied with equality). We restrict to this boundary and minimise.

Substituting $x_2 = 2 - x_1$ into f:

$$f(x_1, 2 - x_1) = (x_1 - 2)^2 + (2 - x_1 - 1)^2 = (x_1 - 2)^2 + (1 - x_1)^2.$$

Expanding: $= x_1^2 - 4x_1 + 4 + x_1^2 - 2x_1 + 1 = 2x_1^2 - 6x_1 + 5$. Setting the derivative to zero: $4x_1 - 6 = 0 \Rightarrow x_1^* = 1.5$. Then $x_2^* = 2 - 1.5 = 0.5$.

$$f(1.5,\, 0.5) = (1.5 - 2)^2 + (0.5 - 1)^2 = 0.25 + 0.25 = 0.5.$$

Classification of Sample Points

Point	**Feasible?**	f **value**	**Note**
$(0, 0)$	Yes	$4 + 1 = 5$	Corner of $\mathcal{X}$

Checking Feasibility Systematically

For any candidate point (x_1, x_2), feasibility requires:

- Constraint g_1: compute

Constrained Optimisation in Python — SciPy Example

Python's `scipy.optimize.minimize` function can solve this problem automatically. The code below sets up the objective, constraints, and bounds, then calls the SLSQP solver (Sequential Least Squares Programming, pronounced "S-L-S-Q-P"), which is designed specifically for smooth constrained problems. SLSQP works by solving a sequence of small quadratic subproblems, each a second-order approximation to the original problem near the current iterate.

Listing 1: Constrained optimization with SciPy

```python
import numpy as np
from scipy.optimize import minimize

# Objective: f(x) = (x1-2)^2 + (x2-1)^2
def f(x):
    return (x[0]-2)**2 + (x[1]-1)**2

# Constraints: x1+x2 <= 2, x1>=0, x2>=0
# SciPy 'ineq' means fun(x) >= 0, so we write
# 2 - x1 - x2 >= 0  <=>  x1+x2 <= 2
constraints = [
    {'type': 'ineq', 'fun': lambda x: 2 - x[0] - x
        [1]},
]
# Bounds encode x1 >= 0 and x2 >= 0
bounds = [(0, None), (0, None)]
```

Reading the SciPy API

- `method='SLSQP'`: the solver algorithm. SLSQP handles both inequality and equality constraints by solving a sequence of quadratic subproblems.
- `bounds`: a list of `(lower, upper)` pairs — one per variable. `None` means no bound on that side, i.e. the variable is unconstrained in that direction.
- `constraints`: a list of dictionaries. `'type': 'ineq'` means $\text{fun}(\mathbf{x}) \geq 0$. This is the *opposite* sign from our $g_i \leq 0$

Outline

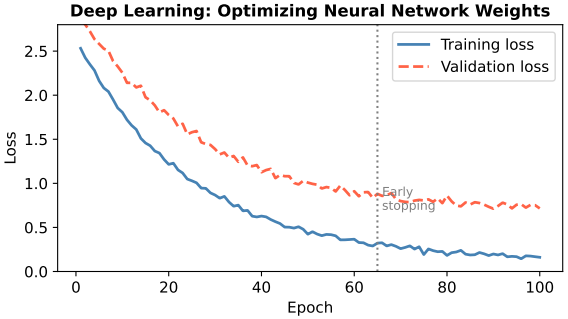
Applications of Optimization I

Optimization pervades nearly every quantitative field. The five examples below show how the same mathematical framework — choose $\mathbf{x}$ (bold x, the design vector) to minimise $f(\mathbf{x})$ subject to constraints — applies across very different domains, from aerodynamics to medicine.

Aircraft Design (Aeronautics)

An aircraft designer wants to minimise aerodynamic drag $D(\mathbf{x})$, which directly determines fuel consumption and operating cost. The design vector $\mathbf{x}$ collects geometric parameters: chord length (the front-to-back distance of the wing cross-section), thickness-to-chord ratio (how thick the wing is relative to its length), and sweep angle (how far back the wing is angled).

The constraints enforce that the wing must generate at least a minimum amount of lift to keep the aircraft airborne, and that structural stresses remain below the material's yield limit. Each evaluation of $D(\mathbf{x})$ requires a Computational



The figure shows typical training and validation loss curves during neural-network optimization. The horizontal axis is the iteration (training step) number; the vertical axis is the loss value $\mathcal{L}$. The training loss (solid line) decreases monotonically as the optimizer

Applications of Optimization II

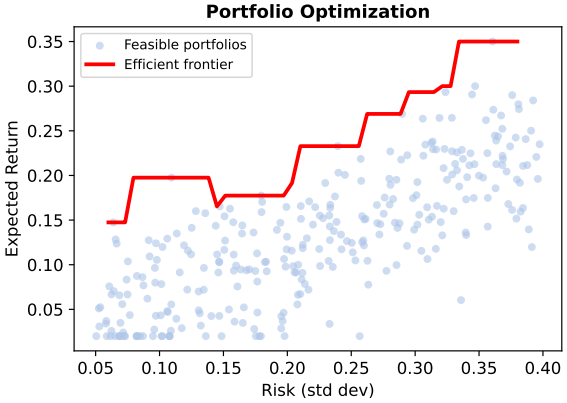
Financial Portfolio Optimization (Markowitz, 1952)

Harry Markowitz showed that portfolio selection can be cast as a quadratic program. Let $\mathbf{x} \in \mathbb{R}^n$ be the vector of portfolio weights, where x_i (x sub i) is the fraction of total wealth invested in asset i.

Objective: minimise portfolio variance (risk):

$$\mathbf{x}^\top \Sigma \mathbf{x}$$

where Σ (Sigma, pronounced "SIG-ma") is the $n \times n$ covariance matrix encoding how assets move together. The notation $\mathbf{x}^\top \Sigma \mathbf{x}$ (x-transpose Sigma x) is called a *quadratic form*: for a two-asset portfolio with $\mathbf{x} = (x_1, x_2)$, it equals $\sigma_{11}x_1^2 + 2\sigma_{12}x_1x_2 + \sigma_{22}x_2^2$, where σ_{ij} (sigma sub



How to Read the Markowitz Formula

The portfolio variance $\mathbf{x}^\top \Sigma \mathbf{x}$ increases when: (1) individual weights x_i are large (concentrated bets)

Applications of Optimization III

Medical Radiation Therapy Planning

In cancer radiotherapy, the goal is to deliver a prescribed radiation dose to the tumour while minimising collateral damage to surrounding healthy tissue. The optimizer chooses the *beam intensities* (how strong each radiation beam is) and *beam angles* (the directions from which beams enter the body), all collected in the design vector $\mathbf{x}$.

The objective is a risk measure over the spatial dose distribution — for example, the Conditional Value-at-Risk (CVaR, pronounced "C-VAR") of the dose received by critical organs such as the spinal cord. Constraints require that the tumour receives at least the prescribed minimum dose everywhere within the tumour volume.

Cross-Domain Summary

Despite their apparent differences, all five applications share the same mathematical skeleton: choose a vector of decision variables to minimise a scalar objective subject to constraints.

Domain	**Minimise**	**Variables**
Aeronautics	Drag $D(\mathbf{x})$	Wing geometr
Deep learning	Loss $\mathcal{L}(\mathbf{x})$	Network weigl
Finance	Portfolio variance	Asset fraction
Medicine	Organ dose risk	Beam intensit
Manufacturing	Makespan	Job–machine

Mastering the standard form unlocks all these applications simultaneously.

Outline

Types of Minima I

Not all "low points" of a function are equally desirable. An algorithm might converge to a point that is locally optimal but globally suboptimal — like resting in a small valley while a much deeper valley exists elsewhere in the landscape. This section defines precisely what we mean by local and global minima, so we can reason clearly about what an algorithm has and has not achieved.

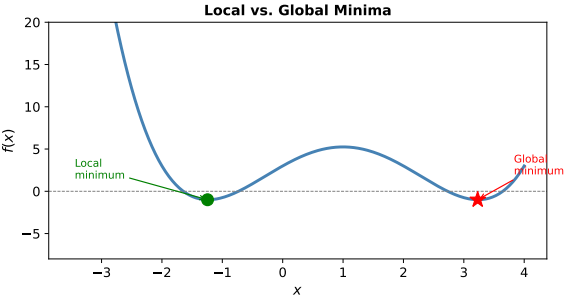
Global Minimum

A point $\mathbf{x}^*$ (x-star, the "starred" or optimal point) is a **global minimum** of f over $\mathcal{X}$ if

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \text{for all } \mathbf{x} \in \mathcal{X}.$$

In plain words: no other feasible point produces a smaller objective value. If the inequality is strict — $f(\mathbf{x}^*) < f(\mathbf{x})$ for all $\mathbf{x} \neq \mathbf{x}^*$ — then $\mathbf{x}^*$ is the *unique* global minimum.

Finding the global minimum of a general non-convex function is NP-hard in the worst case — no polynomial-time algorithm can guarantee



Concrete Numerical Example

Consider $f(x) = \frac{x^3}{3} - x$ ("x-cubed over three minus

Types of Minima II

Classification of Stationary Points

A **stationary point** is any point where the gradient is zero (or, in one dimension, where $f'(x) = 0$). Not all stationary points are minima:

Type	**Condition on f**	**Uniqueness**	**Remark**
Strong local min	$f(\mathbf{x}^*) < f(\mathbf{x})$ for nearby $\mathbf{x} \neq \mathbf{x}^*$	Locally unique	Most algorithms find the
Weak local min	$f(\mathbf{x}^*) \leq f(\mathbf{x})$ for nearby $\mathbf{x}$	Plateau possible	Flat regions complicate s
Global min	$f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{X}$	May be non-unique	Hard to certify in general
Saddle point	Gradient zero but not a min or max	N/A	Gradient descent can sta

Types of Minima III

Practical Implication: Most Algorithms Find Local Minima Only

The vast majority of practical optimization algorithms — gradient descent, Newton's method, quasi-Newton, conjugate gradients — are designed to find *local* minima. If the objective is non-convex, they may converge to a local minimum that is far from the global optimum.

Common strategies to search for a global minimum:

- **Multi-start**: run the algorithm from many random initial points and keep the best solution found. Simple but requires many evaluations.
- **Simulated annealing**: occasionally accept a worse solution with a probability that decreases over time, allowing the search to "climb out" of local basins. The name comes from metallurgy (cooling metal slowly lets atoms find low-energy crystalline arrangements).
- **Population methods** (genetic algorithms, particle swarm): maintain a diverse set of solutions to explore multiple regions simultaneously.
- **Convex relaxation**: reformulate the problem to remove non-convexities, obtaining a global lower bound and often a good solution.

Types of Minima IV

Exercise 1.1 (Solved)

Question: Give a function that has a local minimum that is not a global minimum.

Answer: $f(x) = x^3/3 - x$. The local minimum at $x = 1$ has value $f(1) = -2/3$, but $f(x) \to -\infty$ as $x \to -\infty$, so no global minimum exists.

Exercise 1.2 (Solved)

Question: What is the minimum of $f(x) = x^3 - x$?

Answer: No minimum exists. As $x \to -\infty$, x^3 dominates and $f(x) \to -\infty$, so the function is unbounded below and therefore has no global minimiser anywhere on $\mathbb{R}$.

Outline

Optimality Conditions — One Variable I

A key question in optimization is: how do we *recognise* a minimum without exhaustively testing every possible point? The answer lies in *optimality conditions* — mathematical properties that every minimum must satisfy. If a point fails these conditions, it cannot be a minimum. We begin with the one-dimensional case for clarity, then generalise to multiple variables in the next section.

Setting: Unconstrained Univariate Minimisation

We consider the problem of minimising a smooth (infinitely differentiable) function $f : \mathbb{R} \to \mathbb{R}$ over the entire real line, with no constraints. Our goal is to find conditions on a candidate point x^* that are *necessary* for x^* to be a local minimum.

First-Order Necessary Condition (FONC)

If x^* is a local minimum of f, then

$$f'(x^*) = 0.$$

Here $f'(x^*)$ (f prime at x-star) is the first derivative of f evaluated at the point x^*. A point where the derivative is zero is called a **stationary point** (the

Sufficient Condition for a Strong Local Minimum

If both of the following hold simultaneously:

$$f'(x^*) = 0 \quad \text{and} \quad f''(x^*) > 0,$$

then x^* is a **strong local minimum**. The strict inequality $f''(x^*) > 0$ ensures that the function

Derivation of the FONC via Taylor Expansion

To understand *why* the FONC must hold, we use the Taylor expansion of f around x^*. For a small step size $h > 0$ (h, a small positive real number representing how far we move from x^*), we have:

$$f(x^* + h) = f(x^*) + h\,f'(x^*) + O(h^2),$$
$$f(x^* - h) = f(x^*) - h\,f'(x^*) + O(h^2).$$

Here $O(h^2)$ ("big-O of h-squared", read "order h-squared") denotes terms that shrink at least as fast as h^2 as $h \to 0$ (i.e., they become negligible for small h).

If x^* is a local minimum, then $f(x^* + h) \geq f(x^*)$ and $f(x^* - h) \geq f(x^*)$ for all sufficiently small $h > 0$. Substituting:

$$f(x^* + h) \geq f(x^*) \implies h\,f'(x^*) + O(h^2) \geq 0.$$
$$f(x^* - h) \geq f(x^*) \implies -h\,f'(x^*) + O(h^2) \geq 0.$$

Dividing both inequalities by $h > 0$ and taking $h \to 0$ (so that the $O(h)$ remainder vanishes):

$$f'(x^*) \geq 0 \quad \text{and} \quad -f'(x^*) \geq 0.$$

Optimality Conditions — Multiple Variables I

The one-dimensional conditions extend naturally to functions of n variables, but the derivative is replaced by the *gradient* (a vector of partial derivatives) and the second derivative is replaced by the *Hessian matrix* (a square matrix of second-order partial derivatives). Both objects are central to every topic in the remainder of this book.

First-Order Necessary Condition (Multivariate FONC)

If $\mathbf{x}^*$ is a local minimum of $f : \mathbb{R}^n \to \mathbb{R}$, then

$$\nabla f(\mathbf{x}^*) = \mathbf{0}.$$

Here ∇ (nabla, written ∇, pronounced "NAB-lah"; also called "del" or "grad") denotes the **gradient operator**. The gradient is the vector of all partial derivatives:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right]^\top.$$

Second-Order Necessary Condition (Multivariate SONC)

If $\mathbf{x}^*$ is a local minimum, then the **Hessian matrix** $\nabla^2 f(\mathbf{x}^*)$ (nabla-squared f, pronounced "hessian") must be *positive semidefinite*:

$$\nabla^2 f(\mathbf{x}^*) \succeq 0.$$

The symbol $\succeq 0$ ("succeeds or equals zero", read "is positive semidefinite") means every eigenvalue of the matrix is ≥ 0. The Hessian is the $n \times n$ matrix of second-order partial derivatives:

Optimality Conditions — Multiple Variables II

How to Read the Multivariate FONC and SONC

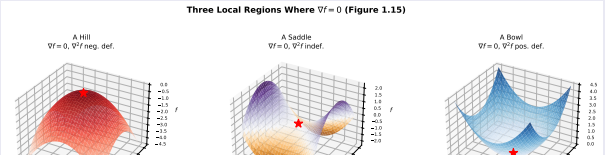
Just as in one dimension, these conditions are *necessary but not sufficient*. Finding a point where $\nabla f = \mathbf{0}$ and $\nabla^2 f \succeq 0$ tells us the point *could* be a local minimum — but it could also be a saddle point (where some eigenvalues of the Hessian are zero). The sufficient condition for a *strong* local minimum is $\nabla f = \mathbf{0}$ *and* $\nabla^2 f \succ 0$ (all eigenvalues strictly positive, read "positive definite").

Optimality Conditions — Multiple Variables III

Three Cases at a Stationary Point ($\nabla f = \mathbf{0}$)

Once we find a stationary point $\mathbf{x}^*$ (satisfying the FONC), the Hessian matrix tells us what kind of stationary point it is. The key is the *signs of the eigenvalues* $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ (lambda sub 1 through lambda sub n) of the Hessian:

1. **Local minimum (bowl shape)**: $\nabla^2 f(\mathbf{x}^*) \succ 0$ (positive definite — all eigenvalues $\lambda_i > 0$). The function curves upward in every direction, so moving away from $\mathbf{x}^*$ in any direction increases f.
2. **Local maximum (hill shape)**: $\nabla^2 f(\mathbf{x}^*) \prec 0$ (negative definite — all eigenvalues $\lambda_i < 0$). The function curves downward in every direction.
3. **Saddle point**: $\nabla^2 f(\mathbf{x}^*)$ is indefinite (some eigenvalues positive, some negative). The function curves upward in some directions and downward in others — like the surface of a horse saddle or a mountain pass. This is *not* a local minimum. Gradient descent can stall near saddle points in high dimensions, which is a significant challenge in deep learning.



Optimality Conditions — Python Verification

The following Python code computes the gradient and Hessian of the Rosenbrock function numerically and verifies that $(1, 1)$ satisfies both the FONC and the SONC. This pattern — compute gradient, compute Hessian, check eigenvalues — is a standard verification workflow that you can apply to any smooth function.

Listing 2: Checking optimality conditions for the Rosenbrock function

```python
import numpy as np

def rosenbrock(x):
    """f(x) = (1-x1)^2 + 5*(x2-x1^2)^2"""
    return (1 - x[0])**2 + 5*(x[1] - x[0]**2)**2

def grad_rosenbrock(x):
    """Gradient vector [df/dx1, df/dx2]"""
    x1, x2 = x
    df_dx1 = 2*(10*x1**3 - 10*x1*x2 + x1 - 1)
    df_dx2 = 10*(x2 - x1**2)
    return np.array([df_dx1, df_dx2])

def hess_rosenbrock(x):
    """2x2 Hessian matrix of second derivatives"""
    x1, x2 = x
    h11 = -20*(x2 - x1**2) + 40*x1**2 + 2
```

Expected Output

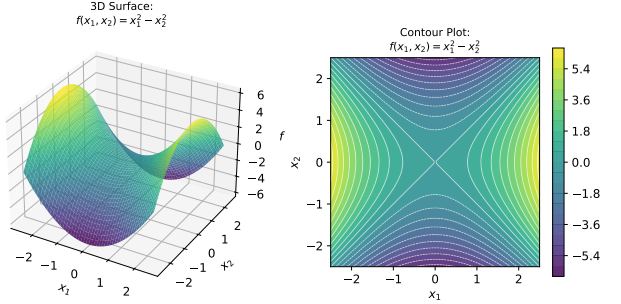
```
Gradient:  [0.  0.]
Hessian:
 [[42.  -20.]
  [-20.  10.]]
Eigenvalues:  [2.0, 50.0]
(both positive)
FONC satisfied:  True
SONC satisfied:  True
```

Conclusion

Both conditions are satisfied at $\mathbf{x}^* = (1, 1)$,

Surface and Contour Visualisation

Visualising the objective function as a 3-D surface and a 2-D contour plot helps build geometric intuition about where minima, maxima, and saddle points are located. The code below generates both views for the saddle function $f(x_1, x_2) = x_1^2 - x_2^2$ (x-one-squared minus x-two-squared), whose origin is a classic saddle point with a positive curvature along x_1 and a negative curvature along x_2.



Listing 3: 3D surface + contour plot

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x1 = np.linspace(-2.5, 2.5, 100)
x2 = np.linspace(-2.5, 2.5, 100)
X1, X2 = np.meshgrid(x1, x2)
Z = X1**2 - X2**2   # saddle function

fig = plt.figure(figsize=(10, 4))

# 3-D surface plot
ax3d = fig.add_subplot(121, projection='3d')
ax3d.plot_surface(X1, X2, Z, cmap='viridis')
ax3d.set_xlabel('$x_1$')
ax3d.set_ylabel('$x_2$')
ax3d.set_title('Surface')
```

Outline

Overview of the Book I

This book is organised around the structure of optimization problems: we begin with the simplest case (one-dimensional, unconstrained, smooth) and progressively add complexity (higher dimensions, constraints, noise, multiple objectives). Understanding where each chapter fits in the overall roadmap will help you navigate the material and know which algorithms apply to your particular problem.

Part I: Foundations (Chapters 1–4)

Ch. 1 **Introduction** (this chapter): standard form, history, applications, local vs. global minima, and optimality conditions (FONC and SONC).

Ch. 2 **Derivatives and Gradients**: how to compute derivatives analytically, numerically via finite differences, and exactly via automatic differentiation — the engine of deep learning.

Ch. 3 **Bracketing**: finding an interval

Part III: Constrained and Specialised (Chapters 10–16)

Ch. 10 **Constraints**: KKT (Karush–Kuhn–Tucker) conditions, penalty methods, augmented Lagrangian, active-set methods, and interior-point methods for constrained optimization.

Ch. 11 **Linear Programming**: the Simplex algorithm and interior-point methods for linear programs.

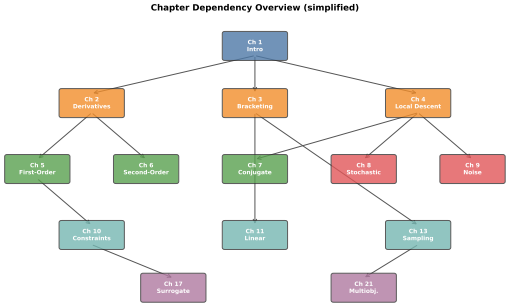
Overview of the Book II

The Progression from Simple to Complex

Chapters 5–9 all assume the objective is smooth and unconstrained (or only box-constrained). They differ in *what information about f they use*, with higher-order information enabling faster convergence but at higher computational cost per iteration.

Chapter 5 uses only the gradient ∇f (nabla f, first-order information). The gradient tells us which way is "downhill" but gives no information about curvature — how steeply downhill. Convergence can be slow near the minimum where the gradient is small.

Chapter 6 adds the Hessian $\nabla^2 f$ (nabla-squared f, second-order information), allowing Newton's method to take steps that account for curvature



The chapter dependency graph shows which earlier chapters are prerequisites for each later chapter. Arrows indicate "should be read before." Chapters 1–4 form the mandatory foundation for all subsequent material. Most chapters in Parts III and IV can be read in any order after the foundations.

Overview of the Book III

When Gradients Are Unavailable: Surrogate and Bayesian Methods

Many real engineering problems do not provide gradients. Either the objective is computed by a legacy simulator that outputs only a single scalar number, or the function is stochastic and the gradients are too noisy to be useful. In these cases, we must rely on *function values alone*.

Surrogate models (Chapter 14): we observe the function at a set of evaluation points, then fit a computationally cheap model (such as a polynomial, a radial basis function network, or a Gaussian process — a type of probabilistic model that quantifies our uncertainty about the function everywhere) to the observed values. We then optimise the cheap surrogate model rather than the expensive true function.

Bayesian optimisation (Chapter 17): we maintain a probabilistic belief (a posterior distribution, using Bayes' theorem to update our belief as we collect data) over the unknown function f. At each step, we choose the next evaluation point by maximising an *acquisition function* $\alpha(\mathbf{x})$ (alpha of x) that balances:

- **Exploitation**: evaluating near the current best estimate of the minimum (low predicted value).
- **Exploration**: reducing uncertainty in regions we have not yet sampled (high posterior variance).

Bayesian optimisation is especially valuable when n (the number of variables) is small but evaluations are extremely expensive.

Outline

Summary of Chapter 1 I

Chapter 1 introduced the fundamental concepts that underpin every topic in this book. Before proceeding to the algorithms, make sure you can state and apply each of the following ideas. If any concept is still unclear, return to the relevant section and work through the examples again.

Core Concepts

1. **What is optimization?** The process of choosing a design vector $\mathbf{x}$ (bold x) from a feasible set $\mathcal{X}$ (script X) to minimise (or maximise) an objective function $f(\mathbf{x})$.
2. **Standard form**: every problem can be written as
$$\min_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}) \quad \text{subject to constraints.}$$
3. **Local minimum**: a point $\mathbf{x}^*$ where $f(\mathbf{x}^*) \le f(\mathbf{x})$ for all nearby $\mathbf{x}$. A **global minimum** is one where this holds for *all* feasible $\mathbf{x}$. Most algorithms find local minima; finding global minima is generally much harder.

Core Formulas to Memorise

FONC (1-D): $f'(x^*) = 0$

FONC (n-D): $\nabla f(\mathbf{x}^*) = \mathbf{0}$

SONC (1-D): $f''(x^*) \ge 0$

SONC (n-D): $\nabla^2 f(\mathbf{x}^*) \succeq 0$

Sufficient: $f'(x^*) = 0,\ f''(x^*) > 0$

In each formula, f' (f prime) denotes the first derivative in 1-D, f'' (f double-prime) the second derivative, ∇ (nabla) the gradient vector in n-D, and ∇^2 (nabla-squared) the Hessian matrix. The symbols $\succeq 0$ and $\succ 0$

Exercises (with Full Solutions) I

The following exercises are drawn directly from the textbook. Work through each one yourself before reading the solution — the process of struggling with a problem and discovering the answer is far more educational than simply reading the answer. Each exercise illustrates a subtle but important point about the nature of optima.

Exercise 1.1

Question: Give an example of a function that has a local minimum that is not a global minimum.

Solution: Consider $f(x) = \frac{x^3}{3} - x$ (one-third x-cubed minus x).

- The derivative is $f'(x) = x^2 - 1$. Setting $f'(x) = 0$ gives $x^2 = 1$, so stationary points are at $x = +1$ and $x = -1$.
- Second derivative: $f''(x) = 2x$. At $x = 1$: $f''(1) = 2 > 0$, so $x = 1$ is a local minimum with $f(1) = \frac{1}{3} - 1 = -\frac{2}{3}$.
- As $x \to -\infty$, $f(x) = \frac{x^3}{3} - x \to -\infty$, so the function is unbounded below. The local minimum at $x = 1$ cannot be a global minimum because there are points with much smaller values.

Exercises (with Full Solutions) II

Exercise 1.2

Question: What is the minimum of $f(x) = x^3 - x$?

Solution: No minimum exists. As $x \to -\infty$, the cubic term x^3 dominates and $f(x) \to -\infty$, so f has no lower bound and therefore no global minimiser on the real line $\mathbb{R}$. Note that this function has a local minimum at $x = 1$ (where $f'(1) = 0$ and $f''(1) = 6 > 0$), but that local minimum is not global.

Exercises (with Full Solutions) III

Exercise 1.3

Question: Give a function that is bounded below but has no global optimum on $\mathcal{X} = [0, 1]$ (the closed interval from 0 to 1).

Solution: Define

$$f(x) = \begin{cases} x & \text{if } x \neq 0 \\ 1 & \text{if } x = 0 \end{cases}.$$

This function satisfies $f(x) > 0$ for all $x \in [0, 1]$ (bounded below by 0). The infimum (greatest lower bound) is $\inf_{x \in [0,1]} f(x) = 0$, but this infimum is never attained: at $x = 0$ the function jumps to 1, and for any $x > 0$ we have $f(x) = x > 0$. So there is no point in $[0, 1]$ where f achieves its infimum of 0. This illustrates that the infimum and minimum are different concepts: the infimum always exists for a bounded-below function, but a minimum (the infimum actually achieved by some feasible point) may not.

Exercises (with Full Solutions) IV

Exercise 1.4

Question: What are the minima and minimizers of $f(x) = \sin(x)$?

Solution: The function $\sin(x)$ is periodic with period 2π ("2 pi", approximately 6.28 radians, one full revolution). Its global minimum value is -1, since $\sin(x) \geq -1$ for all x and $\sin(x) = -1$ whenever $x = \frac{3}{2}\pi + 2\pi k$ for any integer k (i.e., at $x \approx 4.71,\ 10.99,\ 17.28,\ \ldots$ and $x \approx -1.57,\ -7.85,\ \ldots$). There are infinitely many global minimizers, all equally optimal. This illustrates that a function can have multiple (even infinitely many) points at which the global minimum is attained.

Exercises (with Full Solutions) V

Exercise 1.5

Question: Does the FONC $f'(x) = 0$ necessarily hold at the optimal solution of a *constrained* problem?

Solution: **No.** Consider the simple problem $\min_x f(x) = x$ subject to $x \geq 1$. The optimal solution is $x^* = 1$ (the smallest feasible value), but $f'(1) = 1 \neq 0$.

The reason is that at a constrained boundary, the optimizer is "pushing against" the constraint: moving away from $x^* = 1$ in the feasible direction ($x > 1$) would increase f, even though the gradient $f'(1) = 1 > 0$ is non-zero. The correct first-order conditions for constrained problems are the **KKT (Karush–Kuhn–Tucker) conditions** (Chapter 10), which generalise the FONC by accounting for the forces exerted by active constraints.

Exercises (with Full Solutions) VI

Exercise 1.6

Question: How many minima does $f(x, y) = x^2 + y$ have, subject to the constraints $x > y \geq 1$?

Solution: **None.** The constraint $x > y$ (strict inequality) excludes the boundary $x = y$ from the feasible set. As we let $x \to y^+$ (x approaches y from above) and $y \to 1^+$ (y approaches 1 from above), $f(x, y) \to 1^2 + 1 = 2$, but we can never actually attain $f = 2$ because the boundary is excluded. Therefore the infimum of f over the feasible set is 2, but no minimiser exists. The problem's solution requires a closed feasible set (containing its boundary); open strict inequalities can prevent a minimum from existing even when the infimum is finite.

Exercises (with Full Solutions) VII

Exercise 1.7

Question: How many inflection points does $f(x) = x^3 - 10$ have?

Solution: An **inflection point** is a point where the second derivative changes sign (the function switches from concave-up to concave-down, or vice versa). Computing:

$$f'(x) = 3x^2, \qquad f''(x) = 6x.$$

Setting $f''(x) = 6x = 0$ gives $x = 0$. For $x < 0$, $f''(x) = 6x < 0$ (concave-down, curving like an arch). For $x > 0$, $f''(x) = 6x > 0$ (concave-up, curving like a bowl). Since the sign of f'' changes at $x = 0$, there is exactly **one inflection point**, located at $x = 0$.

The -10 is merely a vertical shift of the entire graph and does not affect the shape, the curvature, or the location of the inflection point. The inflection point is at $(0,\ f(0)) = (0, -10)$.

Exercises (with Full Solutions) VIII

Study Tips for Chapter 1

- Always check *both* the FONC *and* the SONC when identifying a minimum. The FONC alone is not sufficient — a stationary point could be a maximum or saddle. For the sufficient condition, also verify that the Hessian is strictly positive definite.
- For constrained problems, the unconstrained FONC ($\nabla f = \mathbf{0}$) does not apply at boundary solutions. The correct framework is the KKT conditions, introduced in Chapter 10. This distinction is crucial for applied work — never apply the unconstrained FONC to a constrained problem without checking feasibility.
- In Python, `scipy.optimize.minimize` is your go-to solver. Use `method='L-BFGS-B'` (L-BFGS-B) for unconstrained or box-constrained problems, and `method='SLSQP'` when you have general inequality or equality constraints.
- Draw the function *before* running an algorithm. For one- and two-dimensional problems, plotting the objective and feasible set often reveals whether local minima exist, where the global minimum is likely to be, and which algorithm family is most appropriate. Visualisation is not optional — it is a diagnostic tool.

Chapter 1: Introduction

Algorithms for Optimization

Kochenderfer & Wheeler, 2nd Ed. (2026)

What We Covered in This Chapter

- A brief history of optimization from ancient Greek geometry to modern deep learning, tracing how each advance was driven by practical necessity.
- The five-step engineering optimization process: specify, formulate, select algorithm, evaluate, iterate.
- Standard mathematical formulation: objective function $f(\mathbf{x})$, design vector $\mathbf{x}$

Up Next: Chapter 2 — Derivatives and Gradients

Before we can use gradient-based algorithms, we need reliable and efficient methods to compute derivatives. Chapter 2 covers:

- Finite difference approximations (forward, central, and backward differences) and their